k results (where k is the number of centroids/reducers) into a single list, that will be fed again to the mappers in the next iteration of the cycle!

Once the cycle is over, we get only centroids as result: we can run the mappers a last time outside of the cycle to get the points assigned to each cluster (in this step, at line #10 of listing 13.2, we can imagine a new set of reducers will be used, dummy pass-through nodes, just returning their input).

This implementation of k-means uses canopy clustering only to bootstrap convergence with a better-than-random initial choice of centroids; distributing the classification and re-centering steps is already a great improvement, and there is a good chance that the improvement you get is already enough to satisfy your requirements.

Still, even if canopy clustering is faster than an iteration of k-means, and can be made even faster by using a cheap approximated metric instead of Euclidean distance, for huge datasets the risk is that you can waste most of the gain obtained by distributing the implementation of k-means through MapReduce, if you run canopy clustering on a single machine; moreover, sometimes this option would not even be available, for huge datasets that won’t fit on any machine.

Luckily for us, we can apply MapReduce to canopy clustering as well!

## 13.3.1 Parallelizing Canopy Clustering